

Contents

Les Dates en Python	3
1 Introduction	3
2 Le module datetime	4
Structure du module	4
3 La classe date	4
4 La classe time	5
5 La classe datetime	5
6 La classe timedelta	5
7 Différence entre deux dates	6
8 Jour de la semaine — weekday()	6
9 Formatage — strftime	7
Parsing — strptime	7
Gestion des erreurs de parsing	8
1 1 Comparaison de dates	8
1 2 Fuseau horaire — timezone	9
Point important	9
1 3 Timestamp Unix	9
1 4 Format ISO 8601	10
1 5 Affichage en français — locale	10
1 6 Cas métier : Calcul d'échéance	11
1 7 Cas métier : Journal d'événements horodaté	11
1 8 Mini-exercices de production	12
Exercice 1 — Prédiction	12
Exercice 2 — Production	12
Exercice 3 — Débogage	13
Synthèse	14
20 Questions Intelligentes — Les Dates en Python	14

1) Différence entre <code>date.today()</code> et <code>datetime.now()</code> ?	14
Énoncé	14
Solution	14
2) Pourquoi <code>datetime.now()</code> et <code>datetime.utcnow()</code> sont différents ?	15
Énoncé	15
Solution	15
3) Pourquoi <code>strptime</code> lève une exception avec un mauvais format ?	15
Énoncé	15
Solution	15
4) Comment calculer l'âge exact ?	16
Énoncé	16
Solution	16
5) Différence entre <code>timedelta(days=1)</code> et <code>timedelta(hours=24)</code> ?	17
Énoncé	17
Solution	17
6) Comment trier une liste de dates ?	17
Énoncé	17
Solution	17
7) Comment obtenir le premier jour du mois ?	18
Énoncé	18
Solution	18
8) Comment trouver le dernier jour du mois ?	18
Énoncé	18
Solution	18
9) Comment vérifier si une année est bissextile ?	19
Énoncé	19
Solution	19
10) Comment afficher une date en français ?	19
Énoncé	19
Solution	19
11) Pourquoi comparer <code>datetime</code> avec <code>date</code> échoue ?	20
Énoncé	20
Solution	20
12) Comment calculer le nombre de jours ouvrables ?	20

Énoncé	20
Solution	20
13) Comment convertir un timestamp Unix en datetime ?	21
Énoncé	21
Solution	21
14) Comment générer toutes les dates d'un mois ?	22
Énoncé	22
Solution	22
15) Pourquoi utiliser isoformat() ?	22
Énoncé	22
Solution	22
16) Comment parser une date ISO 8601 ?	23
Énoncé	23
Solution	23
17) Comment mesurer le temps d'exécution ?	23
Énoncé	23
Solution	23
18) Différence entre replace() et timedelta ?	24
Énoncé	24
Solution	24
19) Pourquoi utiliser pendulum en production ?	24
Énoncé	24
Solution	24
20) Cas métier : Rapport d'activité mensuel	25
Énoncé	25
Solution	25

Les Dates en Python

Le support est propriété de **Bechir Bejaoui**

1 Introduction

La gestion des dates est une compétence fondamentale :

- Horodatage des événements

- Calcul de délais et d'échéances
- Formatage pour affichage ou export
- Comparaison et tri chronologique

Python propose le module **datetime** comme outil central.

2 Le module datetime

Structure du module

```
"""
Le module datetime contient plusieurs classes :
- date      → date seule (année, mois, jour)
- time      → heure seule (heure, minute, seconde)
- datetime  → date + heure combinées
- timedelta → durée / intervalle
- timezone  → gestion des fuseaux horaires
"""

from datetime import date, time, datetime, timedelta, timezone
```

3 La classe date

```
"""
Représente une date (année, mois, jour).
"""

from datetime import date

d = date(2024, 6, 15)
print(d)
print(d.year)
print(d.month)
print(d.day)

aujourd_hui = date.today()
print("Aujourd'hui :", aujourd_hui)
```

4 La classe time

```
"""
Représente une heure seule, sans date associée.
Utile pour des plages horaires ou des horaires fixes.
"""

from datetime import time

t = time(14, 30, 0)
print(t)
print(t.hour)
print(t.minute)
print(t.second)

# Heure minimale et maximale
print("Min :", time.min)
print("Max :", time.max)
```

5 La classe datetime

```
"""
Représente une date et une heure combinées.
"""

from datetime import datetime

maintenant = datetime.now()
print("Maintenant :", maintenant)

dt = datetime(2024, 6, 15, 14, 30, 0)
print("Date fixe :", dt)
print("Heure :", dt.hour)
print("Minute :", dt.minute)
```

6 La classe timedelta

```
"""
Représente une durée.
Permet les calculs de dates.
"""
```

```

from datetime import date, timedelta

aujourd'hui = date.today()

dans_10_jours = aujourd'hui + timedelta(days=10)
il_y_a_5_jours = aujourd'hui - timedelta(days=5)

print("Dans 10 jours :", dans_10_jours)
print("Il y a 5 jours :", il_y_a_5_jours)

```

7 Différence entre deux dates

```

"""
La soustraction de deux dates retourne un timedelta.
"""

from datetime import date

naissance = date(1990, 5, 20)
aujourd'hui = date.today()

difference = aujourd'hui - naissance

print("Jours vécus          :", difference.days)
print("Années approximatives :", difference.days // 365)

```

8 Jour de la semaine — weekday()

```

"""
weekday() retourne un entier :
0 = Lundi, 1 = Mardi, ..., 6 = Dimanche
Utile pour filtrer les jours ouvrables.
"""

from datetime import date

d = date.today()

print("Numéro du jour :", d.weekday())

jours = ["Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche"]

```

```

print("Jour          :", jours[d.weekday()])

# Tester si c'est un jour ouvrable
if d.weekday() < 5:
    print("Jour ouvrable")
else:
    print("Week-end")

```

9 Formatage — strftime

strftime convertit une date en chaîne selon un format.

```

"""
Formatage d'une date en chaîne lisible.
"""

from datetime import datetime

maintenant = datetime.now()

print(maintenant.strftime("%d/%m/%Y"))
print(maintenant.strftime("%d/%m/%Y %H:%M:%S"))
print(maintenant.strftime("%A %d %B %Y"))

```

Codes fréquents :

Code	Signification
%Y	Année 4 chiffres
%m	Mois 2 chiffres
%d	Jour 2 chiffres
%H	Heure (24h)
%M	Minutes
%S	Secondes
%A	Nom du jour
%B	Nom du mois

Parsing — strptime

strptime convertit une chaîne en datetime.

```

"""
Conversion d'une chaîne en objet datetime.

```

```
Le format doit correspondre exactement à la chaîne.
"""
```

```
from datetime import datetime

chaine = "15/06/2024"
dt = datetime.strptime(chaine, "%d/%m/%Y")

print(dt)
print(type(dt))
```

Gestion des erreurs de parsing

```
Un format incorrect provoque une ValueError.
Toujours protéger le parsing en production.
"""
```

```
from datetime import datetime

def parser_date(chaine, format):
    try:
        return datetime.strptime(chaine, format)
    except ValueError as e:
        print("Erreur de parsing :", e)
        return None

print(parser_date("2024-06-15", "%d/%m/%Y"))
print(parser_date("15/06/2024", "%d/%m/%Y"))
```

1 1 Comparaison de dates

```
Les dates supportent les opérateurs de comparaison.
"""
```

```
from datetime import date

d1 = date(2024, 1, 1)
d2 = date(2024, 12, 31)

print(d1 < d2)
print(d1 == d2)
print(d2 > d1)
```

1 2 Fuseau horaire — timezone

```
"""
Utiliser timezone pour les dates conscientes du fuseau horaire.
Depuis Python 3.12, datetime.utcnow() est déprécié.
Préférer datetime.now(timezone.utc).
"""
```

```
from datetime import datetime, timezone, timedelta

utc_now = datetime.now(timezone.utc)
print("UTC      :", utc_now)

paris = timezone(timedelta(hours=1))
paris_now = datetime.now(paris)
print("Paris   :", paris_now)
```

Point important

```
"""
Une date sans fuseau horaire est dite "naive".
Une date avec fuseau est dite "aware".
Ne jamais comparer une date naive avec une date aware.
"""
```

```
from datetime import datetime, timezone

naive = datetime.now()
aware = datetime.now(timezone.utc)

try:
    print(naive < aware)
except TypeError as e:
    print("Erreur :", e)
```

1 3 Timestamp Unix

```
"""
Conversion timestamp Unix <-> datetime.
Utile pour les APIs et bases de données.
"""
```

```

from datetime import datetime

maintenant = datetime.now()
ts = maintenant.timestamp()
print("Timestamp :", ts)

retour = datetime.fromtimestamp(ts)
print("Retour      :", retour)

```

1 4 Format ISO 8601

```

"""
isoformat retourne le format standard ISO 8601.
Recommandé pour les APIs et échanges de données.
fromisoformat parse ce format depuis Python 3.7.
"""

from datetime import datetime

maintenant = datetime.now()
print("ISO :", maintenant.isoformat())

dt = datetime.fromisoformat("2024-06-15T14:30:00")
print("Depuis ISO :", dt)

```

1 5 Affichage en français — locale

```

"""
setlocale configure la langue pour strftime.
Le résultat dépend de l'environnement d'exécution.
"""

import locale
from datetime import datetime

try:
    locale.setlocale(locale.LC_TIME, "fr_FR.UTF-8")
    print(datetime.now().strftime("%A %d %B %Y"))
except locale.Error:
    print("Locale fr_FR non disponible sur cet environnement")

```

```
print(datetime.now().strftime("%Y-%m-%d"))
```

1 6 Cas métier : Calcul d'échéance

```
"""
Objectif :
Calculer une date d'échéance à partir d'une date de départ.
Vérifier si l'échéance est dépassée.
"""

from datetime import date, timedelta

date_contrat = date(2024, 1, 15)
duree_jours = 90

echeance = date_contrat + timedelta(days=duree_jours)
aujourd_hui = date.today()
retard = aujourd_hui - echeance

print("Contrat signé le :", date_contrat)
print("Échéance          :", echeance)

if aujourd_hui > echeance:
    print("Retard de", retard.days, "jours")
else:
    print("Contrat en cours")
```

1 7 Cas métier : Journal d'événements horodaté

```
"""
Objectif :
Horodater automatiquement des événements.
"""

from datetime import datetime

def log_evenement(message):
    ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"[{ts}] {message}")

log_evenement("Démarrage application")
```

```
log_evenement("Connexion utilisateur")
log_evenement("Export données")
```

1 8 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
from datetime import date, timedelta

d = date(2024, 1, 31)
print(d + timedelta(days=1))
print(d.weekday())

from datetime import time
t = time(23, 59, 59)
print(t > time(12, 0, 0))
```

Exercice 2 — Production

Écrire un programme qui :

1. Demande une date de naissance au format JJ/MM/AAAA
2. Calcule l'âge exact
3. Calcule le nombre de jours avant le prochain anniversaire
4. Affiche un message formaté avec f-string

Votre solution ici

Solution

```
"""
Calcul d'âge et prochain anniversaire.
"""

from datetime import date, datetime

chaine = input("Date de naissance (JJ/MM/AAAA) : ")
naissance = datetime.strptime(chaine, "%d/%m/%Y").date()

aujourd_hui = date.today()
age = aujourd_hui.year - naissance.year

if (aujourd_hui.month, aujourd_hui.day) < (naissance.month, naissance.day):
```

```

    age -= 1

prochain = naissance.replace(year=aujourd_hui.year)
if prochain < aujourd_hui:
    prochain = prochain.replace(year=aujourd_hui.year + 1)

jours_restants = (prochain - aujourd_hui).days

print(f"Âge                : {age} ans")
print(f"Prochain anniversaire : {prochain.strftime('%d/%m/%Y')}")
print(f"Jours restants       : {jours_restants} jours")

```

Exercice 3 — Débogage

Ce code contient 3 erreurs. Les identifier et les corriger.

```

from datetime import date, datetime, timezone

d = date.today()
dt = datetime.now(timezone.utc)
print(d == dt)

chaine = "2024/06/15"
dt2 = datetime.strptime(chaine, "%d/%m/%Y")

ts = "1718467200"
retour = datetime.fromtimestamp(ts)
print(retour)

```

Solution

```

# Erreur 1 : comparaison date vs datetime + types incompatibles
# Erreur 2 : format strptime ne correspond pas à la chaîne
# Erreur 3 : fromtimestamp attend un float/int, pas une chaîne

from datetime import date, datetime, timezone

d = date.today()
dt = datetime.now(timezone.utc)
print(d == dt.date())    # comparer même type

chaine = "2024/06/15"
dt2 = datetime.strptime(chaine, "%Y/%m/%d")    # format corrigé

ts = 1718467200    # int, pas chaîne
retour = datetime.fromtimestamp(ts)

```

```
print(retour)
```

Synthèse

Classe	Usage
<code>date</code>	Date seule (année, mois, jour)
<code>time</code>	Heure seule
<code>datetime</code>	Date + heure combinées
<code>timedelta</code>	Durée / calcul d'intervalle
<code>timezone</code>	Gestion des fuseaux horaires

- `strftime` convertit une date en chaîne
 - `strptime` convertit une chaîne en date — format doit correspondre exactement
 - `weekday()` retourne 0 (Lundi) à 6 (Dimanche)
 - `isoformat()` est le standard pour les APIs et échanges de données
 - Depuis Python 3.12, `utcnow()` est déprécié — utiliser `datetime.now(timezone.utc)`
 - Ne jamais comparer une date naïve avec une date aware
-

20 Questions Intelligentes — Les Dates en Python

1) Différence entre `date.today()` et `datetime.now()` ?

Énoncé

Comprendre le niveau de précision de chaque méthode.

Solution

```
"""
```

```
Explication :
```

```
date.today() retourne la date du jour sans heure.
```

```
datetime.now() retourne la date et l'heure complète.
"""
```

```
from datetime import date, datetime

print(date.today())
print(datetime.now())
```

2) Pourquoi datetime.now() et datetime.utcnow() sont différents ?

Énoncé

Comprendre la gestion des fuseaux horaires.

Solution

```
"""
Explication :
now() retourne l'heure locale.
utcnow() retourne l'heure UTC mais est déprécié depuis Python 3.12.
Préférer datetime.now(timezone.utc) en production.
"""
```

```
from datetime import datetime, timezone

print(datetime.now())
print(datetime.now(timezone.utc))
```

3) Pourquoi strptime lève une exception avec un mauvais format ?

Énoncé

Gérer les erreurs de parsing.

Solution

```
"""
Explication :
strptime est strict sur le format.
"""
```

```
Une incohérence provoque ValueError.  
Toujours protéger le parsing en production.  
"""
```

```
from datetime import datetime  
  
try:  
    dt = datetime.strptime("2024-06-15", "%d/%m/%Y")  
except ValueError as e:  
    print("Erreur :", e)
```

4) Comment calculer l'âge exact ?

Énoncé

Éviter l'approximation avec `// 365`.

Solution

```
"""  
Explication :  
Comparer mois et jour pour calculer l'âge exact.  
La division par 365 est incorrecte pour les années bissextiles.  
"""  
  
from datetime import date  
  
def calculer_age(naissance):  
    aujourd_hui = date.today()  
    age = aujourd_hui.year - naissance.year  
  
    if (aujourd_hui.month, aujourd_hui.day) < (naissance.month, naissance.day):  
        age -= 1  
  
    return age  
  
print(calculer_age(date(1990, 6, 15)))
```

5) Différence entre timedelta(days=1) et timedelta(hours=24) ?

Énoncé

Observer comportement identique.

Solution

```
"""
Explication :
Les deux représentent la même durée.
timedelta stocke en jours, secondes et microsecondes.
"""

from datetime import timedelta

td1 = timedelta(days=1)
td2 = timedelta(hours=24)

print(td1 == td2)
print(td1.total_seconds())
```

6) Comment trier une liste de dates ?

Énoncé

Utiliser sorted sur des dates.

Solution

```
"""
Explication :
Les dates supportent les comparaisons.
sorted fonctionne directement.
"""

from datetime import date

dates = [date(2024, 3, 1), date(2024, 1, 15), date(2024, 6, 30)]
tries = sorted(dates)

print(tries)
```

7) Comment obtenir le premier jour du mois ?

Énoncé

Remplacer le jour par 1.

Solution

```
"""
Explication :
replace() permet de modifier un attribut de date
sans créer la date manuellement.
"""

from datetime import date

aujourd_hui = date.today()
premier_jour = aujourd_hui.replace(day=1)

print(premier_jour)
```

8) Comment trouver le dernier jour du mois ?

Énoncé

Utiliser calendar.

Solution

```
"""
Explication :
monthrange retourne le premier jour et le nombre de jours du mois.
"""

import calendar
from datetime import date

aujourd_hui = date.today()
dernier_jour_num = calendar.monthrange(aujourd_hui.year, aujourd_hui.month)[1]
dernier_jour = aujourd_hui.replace(day=dernier_jour_num)

print(dernier_jour)
```

9) Comment vérifier si une année est bissextile ?

Énoncé

Utiliser calendar.

Solution

```
"""
Explication :
isleap retourne True si l'année est bissextile.
Une année bissextile a 366 jours.
"""

import calendar

print(calendar.isleap(2024))
print(calendar.isleap(2025))
```

10) Comment afficher une date en français ?

Énoncé

Utiliser locale.

Solution

```
"""
Explication :
setlocale configure la langue pour strftime.
Protéger avec try/except car la locale dépend de l'environnement.
"""

import locale
from datetime import datetime

try:
    locale.setlocale(locale.LC_TIME, "fr_FR.UTF-8")
    print(datetime.now().strftime("%A %d %B %Y"))
except locale.Error:
    print("Locale fr_FR non disponible")
    print(datetime.now().strftime("%Y-%m-%d"))
```

11) Pourquoi comparer datetime avec date échoue ?

Énoncé

Observer l'incompatibilité de types.

Solution

```
"""
Explication :
date et datetime sont deux types différents.
La comparaison directe lève TypeError.
Utiliser .date() pour extraire la partie date d'un datetime.
"""

from datetime import date, datetime

d = date.today()
dt = datetime.now()

try:
    print(d == dt)
except TypeError as e:
    print("Erreur :", e)

print(d == dt.date())    # correct
```

12) Comment calculer le nombre de jours ouvrables ?

Énoncé

Itérer en excluant samedi et dimanche.

Solution

```
"""
Explication :
weekday() retourne 0-4 pour lundi-vendredi.
On itère et on compte uniquement les jours < 5.
"""
```

```

"""

from datetime import date, timedelta

def jours_ouvrables(debut, fin):
    compte = 0
    courant = debut
    while courant <= fin:
        if courant.weekday() < 5:
            compte += 1
        courant += timedelta(days=1)
    return compte

debut = date(2024, 1, 1)
fin = date(2024, 1, 31)

print(jours_ouvrables(debut, fin))

```

13) Comment convertir un timestamp Unix en datetime ?

Énoncé

Conversion depuis un timestamp.

Solution

```

"""
Explication :
fromtimestamp convertit en datetime local.
Attend un int ou float, pas une chaîne.
"""

from datetime import datetime

ts = 1718467200
dt = datetime.fromtimestamp(ts)

print(dt)

```

14) Comment générer toutes les dates d'un mois ?

Énoncé

Itération sur un mois complet.

Solution

```
"""
Explication :
Utiliser timedelta pour incrémenter jour par jour.
monthrange donne le nombre exact de jours.
"""

from datetime import date, timedelta
import calendar

annee    = 2024
mois     = 6
debut    = date(annee, mois, 1)
nb_jours = calendar.monthrange(annee, mois)[1]

for i in range(nb_jours):
    print(debut + timedelta(days=i))
```

15) Pourquoi utiliser isoformat() ?

Énoncé

Format standard pour export.

Solution

```
"""
Explication :
isoformat retourne le format ISO 8601.
Standard universel pour APIs et bases de données.
"""

from datetime import datetime

print(datetime.now().isoformat())
```

16) Comment parser une date ISO 8601 ?

Énoncé

Depuis Python 3.7.

Solution

```
"""
Explication :
fromisoformat parse les dates ISO 8601.
Plus simple que strptime pour ce format standard.
"""

from datetime import datetime

dt = datetime.fromisoformat("2024-06-15T14:30:00")
print(dt)
```

17) Comment mesurer le temps d'exécution ?

Énoncé

Utiliser datetime pour benchmarking simple.

Solution

```
"""
Explication :
Soustraire deux datetime.now() pour obtenir une durée.
Pour des mesures précises, préférer le module time.perf_counter().
"""

from datetime import datetime

debut = datetime.now()

total = sum(range(1_000_000))

fin = datetime.now()

print("Durée :", fin - debut)
```

18) Différence entre replace() et timedelta ?

Énoncé

Comparer deux approches de modification.

Solution

```
"""
Explication :
replace modifie un attribut précis sans calcul.
timedelta ajoute ou soustrait une durée.
"""

from datetime import date, timedelta

d = date(2024, 6, 15)

print(d.replace(month=1))
print(d - timedelta(days=165))
```

19) Pourquoi utiliser pendulum en production ?

Énoncé

Présenter la bibliothèque avancée.

Solution

```
"""
Explication :
pendulum simplifie la gestion des fuseaux horaires.
dateutil est une alternative plus légère.
Plus ergonomiques que datetime pour des besoins avancés.

Installation : pip install pendulum
"""

# import pendulum
# now = pendulum.now("Europe/Paris")
# print(now)
# print(now.in_timezone("America/New_York"))
```

20) Cas métier : Rapport d'activité mensuel

Énoncé

Générer un récapitulatif sur le mois en cours.

Solution

```
"""
Explication :
Combiner date, timedelta et strftime
pour produire un rapport professionnel.
"""

from datetime import date, timedelta
import calendar

aujourd_hui = date.today()
premier      = aujourd_hui.replace(day=1)
nb_jours     = calendar.monthrange(aujourd_hui.year, aujourd_hui.month)[1]
dernier      = aujourd_hui.replace(day=nb_jours)

print("=== Rapport mensuel ===")
print(f"Période      : {premier.strftime('%d/%m/%Y')} → {dernier.strftime('%d/%m/%Y')}")
print(f"Jours total   : {nb_jours}")
print(f"Généré le      : {aujourd_hui.strftime('%d/%m/%Y')}")
```